

INDUSTRIAL PROBLEM TASK

Q6. ALL-PAIRS VS SINGLE-SOURCE SHORTEST PATH

1. Introduction

Shortest-path algorithms are fundamental to computer networks, transportation systems, routing applications, navigation systems, and distributed computing. A shortest path represents a minimum-cost route between two vertices of a weighted graph. Depending on the application, a system may need the shortest paths from one selected source to all other vertices, or it may need the shortest paths between every possible pair of vertices.

Two important algorithms for these requirements are Dijkstra's algorithm and the Floyd–Warshall algorithm. Dijkstra's algorithm is mainly designed for the single-source shortest-path problem, whereas Floyd–Warshall directly solves the all-pairs shortest-path problem. Both algorithms can produce correct shortest distances for a graph with positive edge weights, but their computational approaches and scalability characteristics are different.

This industrial problem analyzes both methods using a sample weighted network. The solution explains the algorithmic steps, computes shortest paths, verifies correctness, compares time and space complexity, and identifies the situations in which each method is most appropriate.

2. Industrial Problem Statement

Consider a network represented by a weighted directed graph. Each vertex represents a network node and each weighted edge represents a communication connection or transportation link with an associated cost. The objective is to determine minimum-cost routes efficiently.

The problem has two possible requirements. In a single-source requirement, a network operator selects one source node and wants the shortest path to every other node. In an all-pairs requirement, the operator wants the shortest path between every pair of nodes. Choosing an algorithm without considering this distinction can cause unnecessary computation and memory consumption.

2.1 Objectives

- Construct a weighted network graph and define its edge costs.
- Compute shortest paths from a selected source using Dijkstra’s algorithm.
- Compute shortest paths between all pairs using Floyd–Warshall.
- Compare the resulting shortest distances and verify correctness.
- Analyze time and space complexity.
- Evaluate suitability for single-source and all-pairs applications.
- Generate scalability insights for large industrial networks.

3. Network Graph Used for Analysis

The following directed weighted network is used as the working example. The vertices are A, B, C, D, and E. The positive weights may represent distance, delay, transmission cost, or another additive routing metric.

The vertex set is $V = \{A, B, C, D, E\}$. Since all edge weights are non-negative, Dijkstra’s algorithm is valid. Floyd–Warshall is also valid and can additionally handle negative edges when there is no negative cycle.

4. Dijkstra’s Algorithm – Single-Source Shortest Path

Dijkstra’s algorithm is a greedy shortest-path algorithm. It begins with one source vertex and progressively finalizes the vertex with the smallest tentative distance. Once a vertex is finalized, its distance is considered permanent under the non-negative-edge-weight assumption.

4.1 Dijkstra Algorithm Steps

1. Select the source vertex A.
2. Set the distance of A to 0 and all other vertices to infinity.
3. Select the unvisited vertex with the smallest tentative distance.
4. Mark that vertex as processed.
5. Relax each outgoing edge by checking whether the route through the selected vertex is shorter.
6. Update the distance and predecessor whenever a shorter route is found.
7. Continue until all reachable vertices have been processed.
8. Use predecessor information to reconstruct the actual shortest paths.

4.2 Dijkstra Calculation from Source A

Initially, the distance to A is 0 and the distances to B, C, D, and E are infinity.

Vertex	Initial Distance
A	0
B	∞
C	∞

D	∞
E	∞

After processing A, $A \rightarrow B = 4$ and $A \rightarrow C = 2$. The smallest unprocessed vertex is C. Processing C gives $B = 2 + 1 = 3$, $D = 2 + 8 = 10$, and $E = 2 + 10 = 12$.

Next, B with distance 3 is processed. $B \rightarrow D$ gives $3 + 5 = 8$, improving D. Finally, D with distance 8 gives $D \rightarrow E = 8 + 2 = 10$, improving E.

Destination	Shortest Distance from A	Shortest Path
A	0	A
B	3	$A \rightarrow C \rightarrow B$
C	2	$A \rightarrow C$
D	8	$A \rightarrow C \rightarrow B \rightarrow D$
E	10	$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E$

5. Floyd–Warshall Algorithm – All-Pairs Shortest Path

Floyd–Warshall is a dynamic-programming algorithm that computes the shortest distance between every pair of vertices. Instead of choosing one source, it considers each vertex as a possible intermediate point and progressively improves the distance matrix.

5.1 Floyd–Warshall Principle

Let $D[i][j]$ represent the current shortest distance from vertex i to vertex j . When vertex k is considered as an intermediate vertex, the algorithm compares the existing path $i \rightarrow j$ with the route $i \rightarrow k \rightarrow j$.

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$

5.2 Initial Distance Matrix

From / To	A	B	C	D	E
A	0	4	2	∞	∞
B	∞	0	1	5	∞
C	∞	1	0	8	10
D	∞	∞	∞	0	2
E	∞	∞	∞	3	0

Zero values on the main diagonal represent zero cost from a vertex to itself. Infinity represents the absence of a direct edge.

5.3 Important Floyd–Warshall Updates

Path	Calculation	Result
$A \rightarrow B$	$A \rightarrow C \rightarrow B = 2 + 1$	3
$A \rightarrow D$	$A \rightarrow C \rightarrow B \rightarrow D = 2 + 1 + 5$	8
$A \rightarrow E$	$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E = 2 + 1 + 5 + 2$	10
$B \rightarrow E$	$B \rightarrow D \rightarrow E = 5 + 2$	7
$C \rightarrow D$	$C \rightarrow B \rightarrow D = 1 + 5$	6
$C \rightarrow E$	$C \rightarrow B \rightarrow D \rightarrow E = 1 + 5 + 2$	8

6. Final Floyd–Warshall Distance Matrix

From / To	A	B	C	D	E
A	0	3	2	8	10
B	∞	0	1	5	7
C	∞	1	0	6	8
D	∞	∞	∞	0	2
E	∞	∞	∞	3	0

The first row of the final Floyd–Warshall matrix gives the shortest distances from A to all other vertices. These values are 0, 3, 2, 8, and 10, which exactly match the Dijkstra results from source A.

7. Correctness Analysis

7.1 Correctness of Dijkstra’s Algorithm

Dijkstra’s algorithm is correct when all edge weights are non-negative. At every iteration, the algorithm selects the unprocessed vertex with the smallest tentative distance. Any alternative route that goes through another unprocessed vertex cannot produce a smaller distance because the selected vertex already has the minimum tentative value. Consequently, once a vertex is finalized, its shortest distance from the source is correct.

7.2 Correctness of Floyd–Warshall

Floyd–Warshall is correct because it systematically considers every vertex as a possible intermediate point. At each stage, the recurrence chooses the smaller of the existing route and the route that passes through the current intermediate vertex. After all vertices have been considered, every possible combination of intermediate vertices has been accounted for. Therefore, the final matrix contains the shortest distance between every pair, provided there is no negative cycle.

8. Time Complexity Analysis

Let V represent the number of vertices and E represent the number of edges. The computational cost differs significantly between the two algorithms.

8.1 Dijkstra Complexity

With an adjacency matrix and simple minimum selection, Dijkstra’s algorithm has $O(V^2)$ time complexity. With an adjacency list and a binary heap priority queue, the complexity becomes $O((V + E) \log V)$. For sparse graphs, where E is much smaller than V^2 , the heap-based version is generally more efficient.

8.2 Floyd–Warshall Complexity

Floyd–Warshall uses three nested loops over the vertices, giving $O(V^3)$ time complexity. This makes it attractive for small or moderate graphs but increasingly expensive for very large graphs.

Algorithm	Time Complexity	Best Use Case
Dijkstra – array/matrix	$O(V^2)$	Small/dense single-source graph
Dijkstra – binary heap	$O((V+E) \log V)$	Large sparse single-source graph
Floyd–Warshall	$O(V^3)$	Small/medium all-pairs graph

9. Space Complexity Analysis

Dijkstra requires a distance array, visited information, and optionally a predecessor array. These require $O(V)$ auxiliary space, in addition to graph storage. With an adjacency list, graph storage is $O(V+E)$.

Floyd–Warshall maintains a distance matrix with one value for every ordered pair of vertices. Consequently, its space requirement is $O(V^2)$, which can become a major limitation for very large networks.

Algorithm	Auxiliary Space	Main Storage
Dijkstra	$O(V)$	Graph + distance/visited structures
Floyd–Warshall	$O(V^2)$	Distance matrix

10. Detailed Comparison

Feature	Dijkstra	Floyd–Warshall
Problem type	Single-source	All-pairs
Approach	Greedy	Dynamic programming
Negative edges	Not supported	Supported if no negative cycle
Time	$O(V^2)$ or $O((V+E)\log V)$	$O(V^3)$
Auxiliary space	$O(V)$	$O(V^2)$
Sparse graph	Highly suitable	Less suitable at large scale
All-pairs output	Repeated runs required	Produced directly
Repeated queries	Requires preprocessing/reruns	Fast lookup after preprocessing
Scalability	Generally better for large sparse graphs	Limited by cubic time/quadratic memory

11. Applicability to Single-Source Problems

Dijkstra is the preferred method when the system needs shortest paths from one source node to many destinations. It avoids calculating paths from unrelated source vertices.

- GPS navigation from the current location to multiple destinations.
- Finding the lowest-cost route from one warehouse to delivery locations.
- Routing packets from one network router to other routers.
- Finding the nearest service center from a selected location.
- Determining minimum-delay routes from one server to other servers.

12. Applicability to All-Pairs Problems

Floyd–Warshall is appropriate when shortest paths are required between every pair of vertices. Once preprocessing is complete, the shortest distance for any pair can be obtained directly from the matrix.

- Constructing a complete routing-cost table.
- Analyzing communication costs between all network nodes.
- Studying transportation networks with a moderate number of locations.
- Performing repeated shortest-path queries over a relatively stable graph.
- Analyzing the complete connectivity and cost structure of a network.

For very large sparse graphs, however, an all-pairs requirement does not automatically mean Floyd–Warshall is the best implementation. Running an efficient single-source shortest-path method from each required source may be preferable depending on graph density, number of queries, and hardware.

13. Scalability Analysis

Scalability describes how an algorithm behaves when the size of the network increases. This is especially important in industrial systems because modern communication networks may contain thousands or millions of nodes.

Floyd–Warshall has $O(V^3)$ time complexity. If the number of vertices increases by a factor of 10, the dominant computational work can increase by approximately a factor of 1000. Its $O(V^2)$ memory requirement also grows rapidly.

Dijkstra scales better for large sparse networks. With a binary heap and adjacency-list representation, its running time is $O((V+E)\log V)$. When the graph is sparse, E is close to V , so the method is much more practical than an $O(V^3)$ approach.

For an all-pairs requirement on a large sparse graph, one possible strategy is to run a suitable single-source algorithm from each required source. This can avoid dense matrix operations, although it remains expensive when every source is required.

14. Industrial Interpretation

In a network-routing environment, Dijkstra is valuable when routing decisions are source-specific. A router or network controller can select one source and calculate minimum-cost routes to its destinations. Link weights may represent latency, congestion cost, distance, energy consumption, or another additive metric.

Floyd–Warshall is valuable for network analysis systems that need a complete view of shortest distances. A network planning tool may need the minimum communication cost between every pair of servers. A precomputed matrix allows repeated queries to be answered quickly.

If link weights change frequently, a precomputed all-pairs matrix may become outdated and require recomputation. In dynamic environments, running a single-source algorithm only for currently needed routes may be more practical.

15. Results Comparison

Destination from A	Dijkstra	Floyd–Warshall	Shortest Path
A	0	0	A
B	3	3	A → C → B
C	2	2	A → C
D	8	8	A → C → B → D
E	10	10	A → C → B → D → E

The identical distances demonstrate consistency between the two methods for the selected source. Floyd–Warshall additionally provides shortest distances from B, C, D, and E to all other vertices, while Dijkstra in this experiment was executed only from A.

16. Advantages and Limitations

16.1 Dijkstra – Advantages

- Efficient for single-source shortest-path problems.
- Effective for sparse graphs with a priority queue.

- Requires relatively low auxiliary memory.
- Simple to implement and interpret.
- Suitable for routing, navigation, and source-specific network analysis.

16.2 Dijkstra – Limitations

- Standard Dijkstra cannot correctly handle negative edge weights.
- An all-pairs solution requires repeated runs.
- Repeated execution may become expensive when every source is required.

16.3 Floyd–Warshall – Advantages

- Directly solves the all-pairs shortest-path problem.
- Simple dynamic-programming recurrence.
- Works with negative edge weights when there is no negative cycle.
- After preprocessing, shortest-distance queries are obtained directly from the matrix.

16.4 Floyd–Warshall – Limitations

- $O(V^3)$ time becomes expensive for large graphs.
- $O(V^2)$ memory can be very large.
- Recomputation may be required when edge weights change.
- Often unnecessary when only one or a few source vertices are needed.

17. Algorithm Selection Guidelines

Requirement	Recommended Method	Reason
One source $\rightarrow$ many destinations	Dijkstra	Avoids unnecessary all-pairs computation
Few selected sources	Dijkstra repeated as needed	Efficient for sparse graphs
Every pair on small/medium graph	Floyd–Warshall	Direct all-pairs matrix
Large sparse network	Dijkstra-based approaches	Better scalability
Negative edges without negative cycles	Floyd–Warshall	Dijkstra is not valid
Many repeated queries on stable moderate graph	Floyd–Warshall	Precomputed matrix gives immediate lookup

18. Conclusion

Dijkstra’s and Floyd–Warshall’s algorithms are both important shortest-path techniques, but they address different computational requirements. Dijkstra uses a greedy strategy to calculate shortest paths from a selected source and is particularly effective for non-negative edge weights. Floyd–Warshall uses dynamic programming to calculate shortest paths between all pairs of vertices.

For the example network, both algorithms produce the same shortest distances from source A: $B = 3$, $C = 2$, $D = 8$, and $E = 10$. This verifies that the two methods are consistent for the selected positive-weight graph.

The major distinction is computational cost. Dijkstra can achieve $O((V+E)\log V)$ with a binary heap and adjacency lists, making it suitable for large sparse networks. Floyd–Warshall requires $O(V^3)$ time

and $O(V^2)$ space, so it is more suitable for smaller or moderate-sized networks where all-pairs results are actually required.

Therefore, the industrial selection can be summarized as: Single-source requirement $\rightarrow$ Dijkstra. All-pairs requirement on a manageable graph $\rightarrow$ Floyd–Warshall. Large sparse network $\rightarrow$ prefer scalable Dijkstra-based or other specialized shortest-path approaches. The final decision should consider graph size, density, edge weights, frequency of queries, network dynamics, and available computational resources.